

CoPa — CorePanel

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	CoPa — CorePanel
Gerente de Projeto	Johann Gossen Ruth
Gerente/Responsável de GCS-DevOps	Johann Gossen Ruth
Fábrica de Software / Squad	Projeto pessoal – desenvolvedor(es): Johann Gossen Ruth
Repositório principal	https://github.com/levirroh/corepanel

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD).	Luiz Camargo
03/09/2026	1.1	Início do preenchimento com base no estado atual do projeto.	Johann Gossen Ruth
07/09/2026	1.2	Atualização do escopo, ferramentas, estrutura do repositório, critérios de qualidade, marcos e dependências com base no projeto CorePanel.	Johann Gossen Ruth

Sumário

(No Word, clique com o botão direito sobre esta área e escolha "Atualizar campo" — ou pressione Ctrl+A e F9 — para gerar o sumário automaticamente.)

Sumário	1
1. Introdução	2
1.1 Finalidade	3
1.2 Escopo	3
1.3 Definições, Acrônimos e Abreviações	3
1.4 Referências	4
1.5 Visão Geral.....	4
2. Cultura DevOps e Papéis na Gerência de Configuração	4
3. Ferramentas, Ambientes e Infraestrutura	5
3.1 Ferramentas do Pipeline	5
3.2 Ambientes e Infraestrutura.....	5
3.3 Estratégia de Branching e Versionamento	6
4. Identificação da Configuração.....	6
4.1 Itens de Configuração.....	6
4.2 Convenção de Nomenclatura	7
4.3 Estrutura do Repositório	7
5. Integração Contínua (CI).....	8
5.1 Etapas do Pipeline de CI	8
5.2 Política de Pull Request e Revisão de Código.....	8
5.3 Critérios de Qualidade (Quality Gate)	9
6. Entrega e Implantação Contínua (CD)	9
6.1 Etapas do Pipeline de CD.....	9
6.2 Estratégias de Implantação	9
6.3 Rollback e Recuperação	10
7. Infraestrutura como Código (IaC).....	10
7.1 Versionamento de Infraestrutura	10
7.2 Ambientes Efêmeros	10
8. Controle de Configuração e Mudança	10
8.1 Processo de Solicitação de Mudança	10
8.2 Comitê de Controle de Mudança (CCM)	10
9. Baselines e Releases	11
9.1 Geração Automática de Baseline	11
9.2 Notas de Versão (Release Notes)	11
10. Estimativa do Status de Configuração.....	11
10.1 Monitoramento e Observabilidade.....	11
10.2 Métricas DORA	11

10.3 Relatórios e Auditorias	11
11. Marcos	12
12. Treinamento e Recursos.....	12
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros	13
14. Quadro-Resumo do Pipeline CI/CD	13

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/indivíduo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado ao desenvolvedor Johann Gossen Ruth, responsável pelo desenvolvimento, operação e manutenção do sistema **CorePanel**. O sistema tem como objetivo representar um painel administrativo para visualização e gerenciamento simulado de dados referentes a usuários, relatórios, registros, atividades recentes e ações do sistema.

O software está em estágio de desenvolvimento contínuo e, no momento, não possui meio público de acesso. A aplicação tem foco em design, estruturação e funcionamento de frontend, utilizando dados estáticos em formato mock para simular o comportamento de um backend. Não há backend real planejado para a versão inicial.

Em suma, o projeto é composto por:

1. **Frontend:** aplicação web desenvolvida com React e TypeScript, com páginas de login, home/dashboard, usuários, relatórios e registros.
2. **Frameworks e bibliotecas:** Vite, React, TypeScript, Tailwind CSS, React Router, Lucide React, Material UI e MUI X Charts.
3. **Dados em estilo mock:** arquivos JSON simulando tabelas de um banco de dados, como usuários, atividades, registros, relatórios e ações.
4. **Camada de serviço simulada:** funções responsáveis por ler os dados mockados e entregar informações prontas para os componentes do frontend.
5. **Documentação:** README, plano de GCS/DevOps e demais documentos de apoio ao projeto.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)

Termo	Significado
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)
Mock	Dados simulados usados para representar respostas de um backend ou banco de dados
Frontend	Camada visual e interativa do sistema executada no navegador

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.
- Repositório do projeto: <https://github.com/levirroh/corepanel>.
- README.md do projeto: ideia, objetivo, stack e funcionalidades.
- Documentação oficial das tecnologias utilizadas no projeto: React, Vite, TypeScript, Tailwind CSS, React Router, Material UI e MUI X Charts.

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps** é **responsabilidade compartilhada entre desenvolvimento e operações** ("**Você constrói, você executa**" — **you build it, you run it**), **apoiada por automação**. Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de <i>branching</i> e versionamento, administra o registro de artefatos/imagens e a IaC.
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados, revisa Pull Requests e é corresponsável pela qualidade do que é implantado.

Papel	Responsabilidades
SRE / Operações <i>Site Reliability Engineering</i>	Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. – <i>Service Level Indicators (SLIs) and Service Level Objectives (SLOs)</i> .
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD.
Segurança (DevSecOps)	Define políticas de análise estática (SAST), análise de dependências (SCA) e gestão de segredos integradas ao pipeline.
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas ou planejadas para o projeto CorePanel:

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Git e GitHub	Versionamento do código-fonte, documentação, histórico de alterações e organização do repositório.
Desenvolvimento frontend	React, TypeScript e Vite	Construção da aplicação web, tipagem estática e ambiente de desenvolvimento rápido.
Estilização	Tailwind CSS	Criação da interface visual, grid, cards, responsividade e identidade visual do painel.
Roteamento	React Router	Navegação entre login, home, usuários, relatórios e registros.
Ícones	Lucide React	Ícones usados na sidebar, cards e botões de ação.
Componentes e gráficos	Material UI e MUI X Charts	Componentes de interface e gráficos para visualização de dados.
Dados mockados	Arquivos JSON	Simulação de tabelas de banco de dados, como usuários, relatórios, registros, atividades e ações.
Gerenciamento de dependências	npm, package.json e package-lock.json	Instalação, travamento e rastreabilidade das bibliotecas usadas.
Qualidade de código	ESLint e TypeScript	Verificação de padrões, erros de tipagem e problemas básicos de código.
CI/CD	GitHub Actions (planejado)	Execução automática de instalação, lint e build a cada Pull Request ou push na branch principal.

3.2 Ambientes e Infraestrutura

Os ambientes devem ser, sempre que possível, provisionados de forma padronizada e rastreável. Como o CorePanel é uma aplicação frontend sem backend real no estágio atual, a infraestrutura é simplificada.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento local	Ambiente executado na máquina do desenvolvedor para implementação e testes manuais.	Execução local com instalação de dependências e servidor de desenvolvimento.
Homologação / Staging	Ambiente planejado para validação da build antes de uma publicação pública. Pode ser feito por preview local ou deploy temporário.	Build concluída com sucesso e validação manual da interface.
Produção	Ambiente futuro de publicação da versão estável do CorePanel, caso o projeto seja disponibilizado publicamente.	Aprovação manual após build, revisão e testes básicos.

[Detalhar por ambiente: DNS, IPs, credenciais de acesso (via cofre de segredos), topologia (load balancer, nós, banco de dados), conforme necessário.]

3.3 Estratégia de Branching e Versionamento

Recomenda-se estratégia de Trunk-Based Development (ou GitHub Flow), com branches de vida curta, para viabilizar integração contínua real:

- main/trunk — sempre implantável (deployable); protegida por regras de PR obrigatório e pipeline verde.
- feature/<descrição-curta> — branches de curta duração (idealmente < 1-2 dias) para cada mudança.
- release/x.y (opcional) — utilizada apenas se houver necessidade de estabilizar uma versão antes do deploy.
- hotfix/<descrição> — correções emergenciais aplicadas diretamente sobre produção e retro-integradas ao trunk.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor...), permitindo geração automática de changelog e da próxima versão pelo pipeline.

4. Identificação da Configuração

4.1 Itens de Configuração

No CorePanel, os principais itens de configuração são:

- Código-fonte da aplicação React/TypeScript.
- Componentes visuais, páginas, serviços e helpers.
- Arquivos JSON usados como banco de dados mockado.
- Arquivos de configuração do projeto, como package.json, package-lock.json, tsconfig, vite.config e configurações do Tailwind.
- Arquivos de assets, ícones ou imagens utilizadas pela interface.
- Scripts de build, lint e execução local.
- Futuras definições de pipeline, como .github/workflows.

- Documentação técnica, README e Plano de Gerenciamento de Configuração.

Itens não aplicáveis no estágio atual:

- Scripts de migração de banco de dados, pois não há banco de dados real.
- Dockerfiles e manifests de containers, pois a aplicação é uma build estática frontend.
- Scripts de IaC, pois não há infraestrutura dedicada provisionada neste momento.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<descrição-curta>	feature/home-dashboard
Branch de correção	fix/<descrição-curta>	fix/login-validation
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(home): adiciona card de ações rápidas
Tag de release	Versionamento Semântico vX.Y.Z	v1.0.0
Componentes React	PascalCase	ActionButton.tsx, NavBarItem.tsx
Services	camelCase + sufixo Service	userService.ts, reportsService.ts
Arquivos JSON mockados	plural em minúsculo	users.json, reports.json, actions.json
Tipos e interfaces	PascalCase	User, Report, DataRecord, Activity

4.3 Estrutura do Repositório

Estrutura prevista/adotada para o CorePanel:

```
corepanel/
  public/
    assets públicos e arquivos estáticos

  src/
    assets/
      imagens, ícones próprios e elementos visuais estáticos

  components/
    componentes reutilizáveis da interface, como NavBar, NavBarItem e ActionButton

  data/
    database/
      users.json
      activities.json
      records.json
      reports.json
      actions.json

  class/
    interfaces e tipos das entidades do mock database

  pages/
```

Login.tsx
Home.tsx
demais páginas do painel

service/
mock/
api/
 userService.ts
 reportsService.ts
 actionsService.ts
 recordsService.ts
 activitiesService.ts

styles/
 arquivos globais de estilo, quando necessários

main.tsx
App.tsx

package.json
package-lock.json
tsconfig.json
vite.config.ts
README.md

Essa estrutura separa os dados simulados, os tipos das entidades, os serviços de leitura dos mocks e os componentes visuais. Assim, a aplicação mantém uma divisão semelhante à de um sistema com backend, mas sem depender de uma API real.

5. Integração Contínua (CI)

Todo Pull Request para a branch principal deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

1. Checkout do código.
2. Instalação/restauração de dependências com npm.
3. Verificação de tipagem com TypeScript.
4. Execução do lint.
5. Build da aplicação com Vite.
6. Verificação de vulnerabilidades em dependências, quando configurada.
7. Geração do artefato estático da aplicação.

5.2 Política de Pull Request e Revisão de Código

- Todo PR deve ser pequeno e com objetivo claro.

- Mudanças de interface, estrutura de dados ou serviços mockados devem ser revisadas antes do merge.
- Merge somente permitido quando a aplicação estiver executando sem erros conhecidos e com build concluída.
- Em projeto individual, a revisão pode ser feita por checklist próprio ou por validação de colega/professor quando necessário.

5.3 Critérios de Qualidade (Quality Gate)

Para o CorePanel, os critérios mínimos de qualidade são:

- O projeto deve instalar dependências sem erro.
- O build da aplicação deve ser concluído com sucesso.
- Não deve haver erros bloqueantes de TypeScript.
- Não deve haver erros bloqueantes de lint.
- As rotas principais devem carregar corretamente.
- A tela de login deve permitir entrada com os usuários mockados definidos.
- A tela Home deve consumir dados vindos dos arquivos JSON e/ou services, evitando números fixos desnecessários.
- Os dados mockados devem manter consistência de relacionamento, por exemplo userId apontando para usuários existentes.
- O layout deve manter funcionamento visual básico em telas desktop e, quando possível, comportamento responsivo.

6. Entrega e Implantação Contínua (CD)

Após a validação do código pelo fluxo de CI, o processo de CD será responsável por disponibilizar a versão estável do CorePanel em ambiente de homologação ou produção. No estágio atual, o deploy público ainda é planejado, não obrigatório.

6.1 Etapas do Pipeline de CD

1. Geração da build estática do frontend.
2. Publicação do artefato em ambiente de homologação ou preview.
3. Validação manual das telas principais: login, home, usuários, relatórios e registros.
4. Aprovação manual para publicação estável.
5. Deploy em ambiente público, caso adotado.
6. Criação de tag de versão e atualização do histórico de release.

6.2 Estratégias de Implantação

Estratégia	Descrição
Deploy estático	Publicação dos arquivos gerados pela build do Vite.
Rolling update	Substituição gradual das instâncias antigas pelas novas.
Blue-Green	Duas versões coexistem e o tráfego é trocado para a nova versão.
Canary Release	A nova versão recebe pequena fração do tráfego real.
Feature Flags	Funcionalidades são implantadas desligadas e ativadas depois.

6.3 Rollback e Recuperação

Toda implantação deve ser reversível. Para o CorePanel, o rollback pode ser realizado retornando para uma tag ou commit anterior no GitHub e publicando novamente a build correspondente. Como o sistema não possui backend ou banco de dados real, o risco de perda de dados é reduzido na versão inicial.

7. Infraestrutura como Código (IaC)

Como o CorePanel é uma aplicação frontend com dados mockados, não há necessidade imediata de infraestrutura complexa ou provisionamento com IaC. Caso o projeto evolua para backend real, banco de dados, autenticação externa ou ambientes em nuvem, a infraestrutura deverá ser descrita como código e versionada no repositório.

7.1 Versionamento de Infraestrutura

- Alterações de infraestrutura seguem o mesmo processo de PR + revisão + pipeline ("plan" automático em PR, "apply" após aprovação).
- O estado da infraestrutura (state file) é armazenado de forma centralizada e versionada, nunca em máquinas locais.

7.2 Ambientes Efêmeros

Sempre que viável, ambientes de revisão (review apps) devem ser provisionados automaticamente por Pull Request, para validação isolada, e destruídos automaticamente após o merge ou fechamento do PR.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo padrão: abertura de item de trabalho → branch de feature → Pull Request → revisão → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

Como o CorePanel é um projeto acadêmico e individual, o CCM será composto pelo próprio desenvolvedor responsável, podendo contar com apoio do professor ou de colegas em mudanças de maior impacto.

O CCM deve ser acionado em situações como:

- Alteração significativa na estrutura geral do projeto.
- Mudança no padrão de dados mockados ou nas entidades principais.
- Inclusão ou remoção de bibliotecas importantes.
- Mudança no fluxo de autenticação ou controle de sessão.
- Publicação de uma versão estável.
- Correções emergenciais que afetem o funcionamento geral da aplicação.

Mudanças simples de interface, textos, estilos, ajustes pequenos ou melhorias incrementais podem seguir apenas o fluxo normal de commit e validação.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Diferente de modelos tradicionais, em que a baseline é criada manualmente em marcos do projeto, neste plano cada implantação bem-sucedida em produção gera automaticamente uma baseline: uma tag de versão (SemVer) associada ao commit, à imagem publicada no registro e ao estado da infraestrutura correspondente.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas automaticamente pelo pipeline a partir dos Conventional Commits desde a última tag, publicadas junto à release no repositório.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Os ambientes de Homologação e Produção devem ser monitorados continuamente (métricas, logs e alertas), permitindo identificar rapidamente falhas decorrentes de uma implantação.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de pipelines (builds, testes, deploys) e os logs do sistema de versionamento constituem, por si, o principal insumo de auditoria de GCS, dispensando relatórios manuais na maior parte dos casos.

11. Marcos

Os principais marcos internos relacionados ao esforço de GCS/DevOps do CorePanel são:

Marco	Descrição	Status
Criação do repositório	Disponibilização do projeto no GitHub.	Planejado/conforme repositório informado
Estrutura inicial do frontend	Criação do projeto React, Vite, TypeScript e Tailwind.	Em andamento
Tela de login	Implementação do login mockado com perfis admin e user.	Em andamento
Home/Dashboard	Implementação da tela inicial com cards, logs, gráficos e ações rápidas.	Em andamento
Mock database	Criação dos arquivos JSON de usuários, registros, relatórios, atividades e ações.	Em andamento
Services mockados	Criação da camada de leitura dos dados simulados.	Em andamento
Build estável	Garantia de build sem erros bloqueantes.	Planejado
Pipeline de CI	Configuração de GitHub Actions para instalar dependências, rodar lint e build.	Planejado
Primeira baseline	Criação da primeira tag estável do projeto.	Planejado
Deploy público	Publicação futura em plataforma estática, se necessário.	Planejado

12. Treinamento e Recursos

Para implementar e manter as práticas descritas neste plano, são necessários os seguintes recursos:

- Conhecimento básico/intermediário em React, TypeScript e componentização.
- Conhecimento em Tailwind CSS para criação da interface e responsividade.
- Uso de Git e GitHub para versionamento, commits, branches e histórico do projeto.
- Noções de CI/CD para configurar futura automação com GitHub Actions.
- Entendimento de dados mockados em JSON e criação de services para simular backend.
- Conhecimento básico em lint, build e validação de dependências.
- Editor de código, como Visual Studio Code.

- Node.js e npm instalados no ambiente de desenvolvimento.
- Navegador para execução e validação visual da aplicação.

Como o projeto é individual, o treinamento ocorre principalmente por estudo prático, consulta à documentação oficial, orientação em aula e revisão contínua do próprio código.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente por meio do `package.json` e do `package-lock.json`.

No CorePanel, as principais dependências de terceiros são:

- React;
- React DOM;
- Vite;
- TypeScript;
- Tailwind CSS;
- React Router;
- Lucide React;
- Material UI;
- MUI X Charts;
- ESLint.

Não há subcontratados ou fornecedores externos desenvolvendo partes do software. Todas as dependências devem ser adicionadas apenas quando tiverem finalidade clara no projeto. Atualizações devem ser realizadas com cuidado, preferencialmente em branch própria, com validação de build e funcionamento visual após a alteração.

Também devem ser observadas práticas como:

- análise automática de vulnerabilidades em dependências, quando configurada;
- atualização controlada de bibliotecas;
- remoção de dependências não utilizadas;
- verificação do impacto de mudanças em versões principais das bibliotecas.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Build, lint, testes unitários/integração, análise de segurança, publicação do artefato/imagem.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch principal.	Aprovação de PR + CI verde
CD — Staging	Deploy automático em homologação + testes de aceitação.	Merge na branch principal
Gate de Release	Aprovação para produção (automática ou manual, conforme criticidade).	Sucesso dos testes em staging
CD — Produção	Deploy (rolling/blue-green/canary), smoke tests, geração de baseline/tag.	Aprovação do gate de release

Nota: este template deve ser adaptado ao contexto real do projeto (ferramentas, criticidade do sistema e maturidade da equipe em DevOps), mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — agora suportados por automação.